



9. Evaluating & Improving Neural Network Models

"The goal of every model out there is generalization."

Previous Section:

 [8. Cross-Validation in Neural Networks](#)

Contents:

[1. Training Set, Validation Set, and Testing Set](#)

[The Three Core Dataset Splits](#)

[Why We Avoid Evaluating on the Training Set](#)

[Different Validation Strategies](#)

[Why Models Make Mistakes](#)

[2. Evaluating the Models](#)

[Training Accuracy vs Validation Accuracy](#)

[Training Loss vs Validation Loss](#)

[MSE and MAE](#)

[Common-Sense Baseline](#)

[Data Leakage](#)

[Important Practical Advice](#)

[3. Improving the Models](#)

[Noise Features](#)

[Random Labels](#)

[Tuning the Learning Rate](#)

[L2 Regularization](#)

[Dropout](#)

[Early Stopping](#)

[Summary](#)

[References](#)

Deep Learning sounds futuristic, almost like some magical form of intelligence. But underneath all the terminology, it still follows the same philosophy as Machine Learning. Deep Learning is not separate from Machine Learning — it is an evolution of it. And like every Machine Learning system ever created, its true purpose is not to impress us on training data, but to perform well on data it has never seen before.

That idea is called **generalization**. A model that performs perfectly only on the data it memorized is not intelligent. It is no different from a student who memorizes answers without understanding concepts. The real challenge begins when the model faces unfamiliar examples. Can it still recognize patterns? Can it still make reliable predictions? That is the moment where learning is truly tested.

But there is another side to this story — **optimization**.

Before a model can generalize, it first needs to learn from the training data itself. During training, the Neural Network continuously adjusts its weights, minimizes loss, and searches for patterns hidden inside the dataset. This process is optimization: improving the model's ability to fit the training data. Without optimization, the model learns nothing. But optimization alone is dangerous, because a model can become *too good* at remembering the training set.

And that is where the balance of Deep Learning begins. A Neural Network can fall into three major states:

- **Underfitting** — the model fails to learn meaningful patterns from the training data.
- **Overfitting** — the model memorizes the training data so heavily that it performs poorly on unseen data.
- **Balanced / Robust Fit** — the model learns enough structure to generalize well while avoiding memorization.

The entire process of evaluating and improving Neural Networks revolves around managing this balance. Not too little learning. Not too much memorization. But a model that understands patterns deeply enough to survive the unknown. Because at the end of the day, the goal of Deep Learning was never just accuracy. It was understanding.

1. Training Set, Validation Set, and Testing Set

If Deep Learning is about generalization, then we immediately run into an important question:

How do we actually know whether a model generalizes well?

Because during training, the model constantly sees data repeatedly. If we evaluate the model using the exact same data it already memorized, then the evaluation becomes meaningless. It is like giving a student the answer sheet before the exam and then being surprised they scored highly.

So in Deep Learning, we split the dataset into different parts, and each part serves a completely different purpose.

The Three Core Dataset Splits

Before training begins, the data is usually divided into: **Training Set**, **Validation Set**, and **Testing Set**. Each one exists for a specific reason.

- **Training Set — “The Learning Phase”**

The training set is the data the model directly learns from. This is where weights are updated, biases are adjusted, gradients are computed, and optimization happens. The Neural Network repeatedly sees these examples during training and gradually learns patterns from them.

You can think of the training set as the model's classroom experience. The model studies these examples again and again, attempting to minimize the loss function and improve its predictions.

- **Validation Set — “The Development Phase”**

Now here is something extremely important: The validation set is **NOT** used to directly train weights and biases. Instead, it is used to evaluate the model *during* development. This distinction matters a lot.

The validation set acts more like a checkpoint system. After training for some time, we test the model on validation data to see how well it performs on unseen examples.

This helps us make important decisions such as:

- Which model architecture performs better?
- How many hidden layers should be used
- Which learning rate works best
- Which batch size is optimal
- When training should stop
- Whether techniques like dropout or L2 regularization are necessary

In other words: The training set teaches the model. The validation set judges the model while it is learning. And this is one of the reasons why validation is so important in Deep Learning. Without it, we would have no reliable way to detect overfitting during development.

• **Test Set — “The Final Exam”**

The test set is used **after training is completely finished**. This data should be entirely new to the model. No learning should happen here. No tuning should happen here. No hyperparameter adjustments should happen here. The test set exists for one purpose only: To estimate how well the model performs in the real world.

This is the closest thing we have to a true evaluation of generalization. Because once a model has indirectly adapted itself to the validation set through repeated experimentation, the test set becomes the only truly untouched dataset remaining.

You can think of it like this:

- Training set → learning
- Validation set → development and tuning
- Test set → final judgment

Why We Avoid Evaluating on the Training Set

Technically, we *can* evaluate the model on the training set. But doing so is often misleading. Why? Because the model has already seen those examples before. A model might achieve extremely high accuracy on training data simply because it memorized patterns rather than learned meaningful relationships.

This is why evaluating only on the training set becomes dangerous. It creates the illusion of intelligence. The model appears confident. The numbers look impressive. But the moment new data appears, performance collapses. That is not learning. That is memorization. And Deep Learning was never meant to be a memory game.

Different Validation Strategies

There are multiple ways to use validation data in practice.

- **Hold-Out Validation** This is the simplest approach. The validation set is completely separated from the training set. For example: 70% training, 15% validation, and 15% testing. The model trains on one portion and validates on another fixed portion. Simple and fast.
- **K-Fold Cross-Validation:** Instead of using only one validation split, we divide the dataset into multiple folds. For example, in 5-Fold Cross-Validation: The dataset is split into 5 parts. One fold becomes the validation set. The remaining folds become the training set. The process repeats until every fold has been used as validation once. This gives a more stable estimate of performance because the model is evaluated across multiple splits rather than relying on a single partition.
- **Iterated K-Fold Validation:** This is simply a repeated version of K-Fold Cross-Validation. The folds are shuffled and recreated multiple times, reducing randomness and giving even more reliable evaluation results. It is computationally more expensive, but often produces stronger estimates of model performance.

Why Models Make Mistakes

Even powerful Neural Networks can fail. And sometimes, the problem is not the model itself — it is the data. Here are some common reasons models make mistakes.

- **Mislabeled Samples:** Sometimes the labels themselves are wrong. A cat image might accidentally be labeled as a dog. The model then learns incorrect relationships from corrupted information.
- **Noise:** Some data contains confusing or meaningless patterns. For example: An image visually looks like the digit "4". But its label says "9". The model receives contradictory signals. Too much noise can severely damage learning.
- **Uncertainty:** Sometimes the data itself is unclear. Examples include blurry images, distorted audio, incomplete text, low-quality recordings. Even humans struggle with uncertain data, so models struggle too.
- **Rare Features:** Some patterns appear very infrequently. For example: certain rare words may appear only in a few reviews, unusual object types may barely exist in training data. Because the model rarely sees them, it struggles to learn reliable relationships.
- **Spurious Correlations:** This is one of the most dangerous problems in Machine Learning. A model sometimes learns patterns that are technically correlated with the data — but completely meaningless in reality. For example: Most dog images were captured outdoors during sunny days. Most cat images were captured indoors at night. The model may accidentally learn: ***sunlight = dog; darkness = cat***. Which is obviously absurd. The model is not truly understanding animals. It is exploiting shortcuts hidden inside the dataset.

And this is why dataset quality matters so much in Deep Learning. Because a Neural Network learns whatever patterns exist — even the bad ones.

2. Evaluating the Models

When people first learn Machine Learning, evaluation often revolves around a familiar collection of metrics: Accuracy, Precision, Recall, F1-Score. And for regression: Mean Squared Error (MSE) and R^2 Score. These metrics are still important in Deep Learning. They did not suddenly disappear. But Deep Learning introduces something much more dynamic.

Unlike many traditional Machine Learning workflows — where we mainly care about the final evaluation on the testing set — Deep Learning focuses heavily on what happens *during training itself*. Why? Because Neural Networks learn progressively over epochs. And during that learning process, many important questions emerge:

- Is the model still learning useful patterns?
- Has the model started memorizing the training data?
- Which epoch produced the best generalization?
- Is the model underfitting or overfitting?
- Should training stop early?
- Should the architecture be changed?

So in Deep Learning, evaluation is not only a “final exam”. It becomes a continuous monitoring process. That is why Training Metrics and Validation Metrics become extremely important.

Training Accuracy vs Validation Accuracy

One of the first things we monitor during training is accuracy. But here is the important part, we do not just track one accuracy. We track two: **Training Accuracy and Validation Accuracy**

- Training Accuracy measures how well the model performs on the data it already learned from.
- Validation Accuracy measures how well the model performs on unseen validation data during development.

And the relationship between these two numbers tells us a lot about the behavior of the model. For example:

- If both accuracies are low → the model is probably underfitting.
- If training accuracy becomes extremely high while validation accuracy stagnates or decreases → the model is likely overfitting.
- If both improve together and remain relatively close → the model is generalizing well.

This is why validation metrics matter so much in Deep Learning. They help us observe the learning behavior itself. Not just the final outcome.

Training Loss vs Validation Loss

In many Deep Learning workflows, loss is often even more important than accuracy. Because accuracy is discrete: correct or incorrect. But loss captures *how wrong* the model is. This gives us a much more detailed picture of learning.

During training, we usually monitor: **Training Loss and Validation Loss**

- The Training Loss measures how well the model minimizes errors on training data.
- The Validation Loss measures how well the model generalizes to unseen validation data.

And interestingly, one of the most important moments in Deep Learning often happens when these two curves begin separating. For example: Training loss continues decreasing and validation loss suddenly starts increasing. This is a classic sign of overfitting. The model is becoming too specialized toward training data and starts losing its ability to generalize.

This is also why Deep Learning practitioners often care deeply about selecting the **best epoch**. Sometimes the best model is not the final epoch. Sometimes the model performs best at Epoch 12, then begins overfitting afterward. So the goal is not: "Train as long as possible". The goal is: "Stop training at the moment where generalization is strongest".

MSE and MAE

For regression tasks, metrics like: Mean Squared Error (MSE) and Mean Absolute Error (MAE) are still heavily used.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

These metrics measure prediction errors numerically:

- MSE penalizes larger errors more heavily because of squaring
- MAE measures average absolute differences directly

They are mostly associated with regression problems such as predicting house prices, forecasting revenue, estimating temperatures, or predicting stock values.

Technically, classification tasks can also use these metrics in some scenarios. But this is generally not the standard practice. In Deep Learning classification problems, metrics tied to probabilities and classification performance are usually preferred.

Common-Sense Baseline

Before celebrating a Neural Network's performance, we should always ask a very simple question: **"Is this model actually better than a trivial solution?"** → This idea is called the **common-sense baseline**.

A common-sense baseline is a very simple prediction strategy used as the minimum reference point for evaluation. If a complicated Deep Learning model cannot outperform a basic baseline, then the model probably has little real value.

Some common baseline examples include:

- **Predicting the Most Common Class.** Suppose: 90% of emails are "not spam". A baseline model could simply predict "not spam" for everything. And surprisingly, it would already achieve 90% accuracy. So if your fancy Neural Network only achieves 85% accuracy, then your model is actually worse than doing almost nothing.
- **Random Guessing:** For balanced binary classification problems, 50% positive and 50% negative. A random guess baseline would achieve around 50% accuracy. Your model should clearly outperform randomness. Otherwise, it has learned almost nothing meaningful.

- **Predicting the Mean or Median:** For regression tasks like house price prediction, a baseline might simply predict the average house price, or the median value for every sample. Simple. Naive. But still important as a reference point.
- **Predicting the Previous Value:** This is common in time-series problems. For example: tomorrow's weather = today's weather, next month's sales = this month's sales. Surprisingly, these naive forecasts can sometimes be difficult to beat.
- **Rule-Based Predictions:** Sometimes a simple hand-crafted rule becomes the baseline. For example: "If email contains 'FREE MONEY', classify as spam". Not intelligent. Not flexible. But useful as a comparison point.

The key idea is simple: ***A Deep Learning model should justify its own complexity.***

Data Leakage

Now we arrive at one of the most dangerous problems in Machine Learning and Deep Learning: **Data Leakage**. Data leakage happens when information from the testing set — or future unseen data — accidentally leaks into training or validation.

And when that happens, evaluation becomes unreliable. The model appears amazing. Metrics become unrealistically high. But the model is essentially cheating. This can happen in many ways: accidentally mixing test samples into training data, preprocessing the entire dataset before splitting, using future information in time-series prediction, or tuning hyperparameters repeatedly based on testing performance.

The danger of data leakage is that it creates a false illusion of generalization. The model does not truly understand unseen data. It simply encountered hidden information beforehand. And this is why dataset separation must be handled extremely carefully.

Important Practical Advice

When evaluating Deep Learning models, remember these principles:

- Split data carefully
- Prevent validation or testing contamination
- Never allow test information to influence training decisions
- Do not rely on a single metric for complex problems

High accuracy alone does not guarantee a good model, especially with imbalanced datasets. A model evaluation should always be interpreted from multiple perspectives. Because Deep Learning is not only about producing high numbers. It is about producing trustworthy generalization.

3. Improving the Models

Earlier, we focused on evaluating Deep Learning models through metrics such as accuracy and loss. But evaluation alone is not enough. A Neural Network can achieve impressive performance during training while still learning the *wrong* patterns underneath. In real-world applications, data is messy, incomplete, noisy, and sometimes even misleading. Because of that, improving a model is not simply about increasing accuracy — it is about making the network learn patterns that actually generalize.

In this section, we experiment with one of the most interesting ideas in Machine Learning and Deep Learning: **noise features**. These are features that carry little to no meaningful information, yet the model may still attempt to learn from them. This experiment demonstrates how Neural Networks can accidentally discover fake relationships inside the data, leading to weaker generalization performance.

To keep our attention on the modeling process itself rather than preprocessing, we will use the famous MNIST dataset.

```

from tensorflow.keras.datasets import mnist

(train_images, train_labels), (test_images, test_labels) =
mnist.load_data()

# Vectorize 28x28 image to 784 channels and normalize to
[0, 1].
train_images = train_images.reshape((60000, 28 * 28)).astype
("float32") / 255.0
test_images = test_images.reshape((10000, 28 * 28)).astype
("float32") / 255.0

print("Train Images:", train_images.shape)
print("Test Images:", test_images.shape)

```

Train Images: (60000, 784)

Test Images: (10000, 784)

Each image originally has a shape of 28×28. After vectorization, every image becomes a vector with 784 numerical features. From here, we intentionally make the dataset *worse* by introducing additional meaningless features.

Noise Features

This experiment is more of a demonstration than an actual improvement technique. We artificially inject extra channels into the dataset and observe how the Neural Network behaves.

We create two new versions of the dataset:

- One version receives **random noise channels**
- Another version receives channels containing only **zeros**

The purpose is to see whether the model struggles more when exposed to meaningless randomness.

```
# Here, we only use 20000 images for demonstration
MNIST_SAMPLE_SIZE = 20000

train_images_demo = train_images[:MNIST_SAMPLE_SIZE]
train_labels_demo = train_labels[:MNIST_SAMPLE_SIZE]

print("Shape train_images_demo:", train_images_demo.shape)
print("Shape train_labels_demo:", train_labels_demo.shape)
```

Shape train_images_demo: (20000, 784)

Shape train_labels_demo: (20000,)

```

import numpy as np

# Randomize 784 noise channels
noise_channels = np.random.random((len(train_images_demo),
784)).astype("float32")

# Create 784 channels with only zeros
zero_channels = np.zeros((len(train_images_demo), 784), dtype="float32")

# Concatenate original data to noise_channels with axis=1.
train_images_with_noise_channels = np.concatenate([train_images_demo,
                                                    noise_channels], axis=1)

# Concatenate original data to zero_channels with axis=1.
train_images_with_zeros_channels = np.concatenate([train_images_demo,
                                                    zero_channels], axis=1)

print("Data shape with noise channels:", train_images_with_noise_channels.shape)
print("Data shape with zero channels:", train_images_with_zeros_channels.shape)

```

Data shape with noise channels: (20000, 1568)

Data shape with zero channels: (20000, 1568)

Notice what happened here. The original dataset had 784 features. After concatenation, the feature space doubles to 1568 dimensions. However, half of those features are completely artificial.

Now we train the same Neural Network architecture on both datasets.

```

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

```

```
# Define the mnist model
def train_mnist_model(input_images):
    input_layer = layers.Input(shape=(input_images.shape
[1],))
    hidden_layer1 = layers.Dense(512, activation="relu")
    hidden_layer2 = layers.Dense(256, activation="relu")
    output_layer = layers.Dense(10, activation="softmax")

    model = keras.Sequential([input_layer, hidden_layer1,
                                hidden_layer2, output_layer])

    model.compile(optimizer="rmsprop", loss="sparse_categorical_crossentropy",
                  metrics=["accuracy"])

    return model
```

- **Training on Noise Channels**

```
# Train the model on the data with noise channels
model_with_noise_channels = train_mnist_model(train_images_
with_noise_channels)

history_noise = model_with_noise_channels.fit(train_images_
with_noise_channels,
                                              train_labels_
demo, epochs=10,
                                              batch_size=12
8, validation_split=0.2, verbose=1)
```

- **Training on Zero Channels**

```
# Train the model on the data with zero channels
model_with_zeros_channels = train_mnist_model(train_images_
with_zeros_channels)
```

```

history_zeros = model_with_zeros_channels.fit(train_images_
with_zeros_channels,
train_labels_
demo, epochs=10,
batch_size=12
8, validation_split=0.2, verbose=1)

```

Finally, we visualize the validation accuracy and validation loss.

```

import matplotlib.pyplot as plt

plt.figure(figsize=(12, 6))

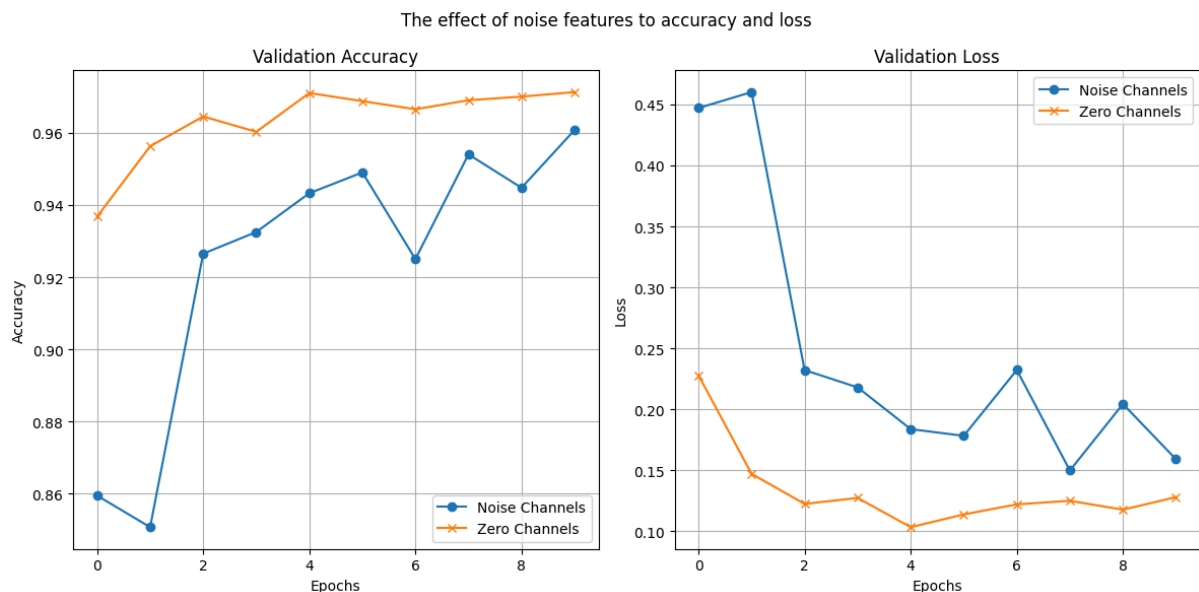
# Plot validation accuracy
plt.subplot(1, 2, 1)
plt.plot(history_noise.history["val_accuracy"], label="Noise Channels", marker='o')
plt.plot(history_zeros.history["val_accuracy"], label="Zero Channels", marker='x')
plt.title("Validation Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

# Plot validation loss
plt.subplot(1, 2, 2)
plt.plot(history_noise.history["val_loss"], label="Noise Channels", marker='o')
plt.plot(history_zeros.history["val_loss"], label="Zero Channels", marker='x')
plt.title("Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

```



```
plt.suptitle('The effect of noise features to accuracy and loss')
plt.tight_layout()
plt.show()
```



To absolutely nobody's surprise, the dataset containing zero channels performs better than the dataset containing random noise channels. Even though both datasets technically contain the same number of additional features, the zero channels are harmless because they do not introduce conflicting patterns. They are constant and predictable.

The random noise channels, however, are chaotic. During training, the Neural Network attempts to search for patterns everywhere — even when no meaningful pattern exists. As a result, the model wastes part of its learning capacity trying to interpret randomness. This weakens generalization and makes learning more unstable.

This phenomenon is closely related to the concept of **spurious correlation**. A Neural Network may accidentally learn fake relationships between meaningless features and target labels simply because those patterns appear temporarily useful during training. In real-life datasets, these fake correlations can become extremely dangerous.

For example:

- A medical model may associate image brightness with disease prediction

- A hiring model may associate irrelevant demographic patterns with success
- A fraud detection system may rely on accidental timing artifacts instead of actual fraud behavior

The model appears accurate during training, but its understanding of the problem is fundamentally flawed.

This is why Deep Learning is not only about building larger Neural Networks. Sometimes, improving a model means carefully removing useless information so the network can focus on the features that truly matter.

Random Labels

Earlier, we introduced meaningless *features* into the dataset and observed how the Neural Network struggled with random noise. Now we perform something even more destructive: we corrupt the **labels themselves**.

This experiment explores a terrifying but important property of Deep Learning models: Neural Networks can memorize surprisingly well — even when the data makes absolutely no sense.

Normally, labels represent meaningful relationships: This image belongs to digit **3**. That image belongs to digit **7**. The model learns patterns connecting the image and its label. But what happens if we completely destroy that relationship?

To test this, we randomly shuffle the labels while keeping the images unchanged.

```
# Create random labels by shuffling the original labels
random_train_labels = train_labels_demo.copy()
np.random.shuffle(random_train_labels)
```

At this point, the dataset becomes fundamentally broken. The images still contain handwritten digits, but the labels are now random. A picture of digit 2 may now be labeled as 8. A digit 5 may become 1. There is no longer a meaningful mapping between inputs and outputs. Yet surprisingly, the Neural Network will still try to learn.

- **Training the Model on Random Labels**

```
# Create random labels by shuffling the original labels
random_train_labels = train_labels_demo.copy()
np.random.shuffle(random_train_labels)

# Train model with the random labels
model_random_labels = train_mnist_model(train_images_demo)
history_random_labels = model_random_labels.fit(train_images_demo, random_train_labels,
                                                epochs=12,
                                                batch_size=128,
                                                validation_split=0.2, verbose=1)
```

Now let us visualize the training process.

```
# Plot
plt.figure(figsize=(12, 6))

# Plot training and validation accuracy
plt.subplot(1, 2, 1)
plt.plot(history_random_labels.history["accuracy"], label="Training Accuracy", marker='o')
plt.plot(history_random_labels.history["val_accuracy"], label="Validation Accuracy", marker='x')
plt.title("Training and Validation Accuracy (Random Labels)")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

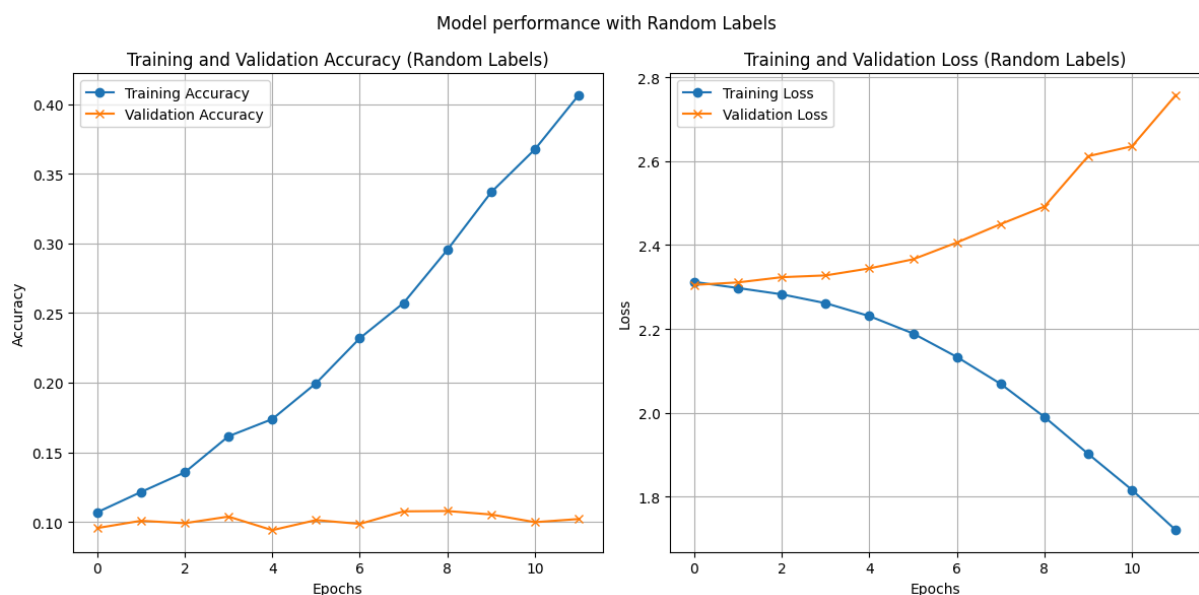
# Plot training and validation loss
plt.subplot(1, 2, 2)
plt.plot(history_random_labels.history["loss"], label="Training Loss", marker='o')
plt.plot(history_random_labels.history["val_loss"], label="Validation Loss", marker='x')
```

```

="Validation Loss", marker='x')
plt.title("Training and Validation Loss (Random Labels)")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.suptitle('Model performance with Random Labels')
plt.tight_layout()
plt.show()

```



To nobody's surprise, again, the validation performance is terrible. The validation accuracy hovers around **10%**, which is essentially random guessing for a 10-class classification problem like MNIST. The model cannot discover meaningful relationships because there are none to discover.

But something very interesting happens during training: The training accuracy slowly increases. The training loss slowly decreases. Why? Because the Neural Network begins to **memorize** parts of the dataset.

Even though the labels are random and meaningless, the model still attempts to associate specific images with specific outputs. Given enough capacity and enough epochs, a Neural Network can eventually memorize almost the entire training set.

This reveals one of the most important truths in Deep Learning: Neural Networks are extremely powerful memorization systems. But memorization is not intelligence. A model that memorizes random labels learns nothing useful about the underlying problem. It cannot generalize because there is no pattern to generalize from. This creates a massive gap between training and validation performance:

- Training performance improves because the model remembers samples
- Validation performance remains terrible because the learned relationships are fake

In practical terms, this model is completely unusable. The experiment also gives us a very clear picture of **underfitting and meaningless learning**. The model is technically learning *something*, but what it learns has no predictive value outside the training data.

This is why data quality matters so much in Deep Learning. You can build the most advanced Neural Network architecture in existence. You can stack hundreds of layers, train for days, and use enormous GPUs. But if the labels themselves are meaningless, the model has nothing meaningful to learn.

The lesson here is simple but extremely important:

- A Neural Network can memorize randomness
- A Neural Network cannot generalize from randomness
- Good performance only comes from meaningful relationships inside the data

Or in simpler terms: ***The data itself must contain truth before the model can learn truth from it.***

Tuning the Learning Rate

One of the most important hyperparameters in Deep Learning is the **learning rate**.

During training, a Neural Network continuously updates its weights and biases in order to reduce prediction error. These updates are performed by optimization algorithms such as **Gradient Descent** and its variants. The learning rate controls *how large* each update step should be.

You can think of it like this:

- A large learning rate takes huge steps toward the solution
- A small learning rate takes tiny, careful steps

Both situations come with advantages and dangers.

- If the learning rate is too high, the optimizer may overshoot the optimal solution repeatedly, causing unstable training. The loss may fluctuate wildly, accuracy may stagnate, and the model can fail to converge properly.
- If the learning rate is too low, training becomes painfully slow. The model improves very gradually and may even become trapped in a local minimum before reaching a better solution.

So choosing a good learning rate becomes a balancing act.

In this experiment, we rewrite the model function so the learning rate can be adjusted manually. Then we compare three different learning rates: 1, 1e-2, and 1e-3. All models are trained for 10 epochs on the same dataset.

- **Rewriting the Model Function**

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Define the mnist model with learning rates
def train_mnist_model_with_lr(input_images, learning_rate):
    input_layer = layers.Input(shape=(input_images.shape
[1],))
    hidden_layer1 = layers.Dense(512, activation="relu")
    hidden_layer2 = layers.Dense(256, activation="relu")
    output_layer = layers.Dense(10, activation="softmax")

    model = keras.Sequential([input_layer, hidden_layer1,
                                hidden_layer2, output_layer])

    optimizer = keras.optimizers.RMSprop(learning_rate=learnin
ng_rate)
    model.compile(optimizer=optimizer, loss="sparse_categorical_crossentropy")
```

```

al_crossentropy",
    metrics=["accuracy"])

return model

```

Now we train three different models using different learning rates.

```

# Train with learning rate=1.0
model_lr_1 = train_mnist_model_with_lr(train_images_demo,
1.0)

history_lr_1 = model_lr_1.fit(train_images_demo, train_labels_demo,
                                epochs=10, batch_size=128, validation_split=0.2, verbose=1)

```

```

# Train with learning rate=1e-2
model_lr_1e2 = train_mnist_model_with_lr(train_images_demo,
1e-2)
history_lr_1e2 = model_lr_1e2.fit(train_images_demo, train_labels_demo,
                                    epochs=10, batch_size=128, validation_split=0.2, verbose=1)

```

```

# Train with learning rate=1e-3
model_lr_1e3 = train_mnist_model_with_lr(train_images_demo,
1e-3)
history_lr_1e3 = model_lr_1e2.fit(train_images_demo, train_labels_demo,
                                    epochs=10, batch_size=128, validation_split=0.2, verbose=1)

```

Finally, we compare the validation accuracy and validation loss.

```

import matplotlib.pyplot as plt

plt.figure(figsize=(12, 6))

# Validation Accuracy
plt.subplot(1, 2, 1)

plt.plot(history_lr_1.history["val_accuracy"], label="LR = 1", marker='o')
plt.plot(history_lr_1e2.history["val_accuracy"], label="LR = 1e-2", marker='x')
plt.plot(history_lr_1e3.history["val_accuracy"], label="LR = 1e-3", marker='s')
plt.title("Validation Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")

plt.legend()
plt.grid(True)

# Validation Loss
plt.subplot(1, 2, 2)

plt.plot(history_lr_1.history["val_loss"], label="LR = 1", marker='o')
plt.plot(history_lr_1e2.history["val_loss"], label="LR = 1e-2", marker='x')
plt.plot(history_lr_1e3.history["val_loss"], label="LR = 1e-3", marker='s')

plt.title("Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")

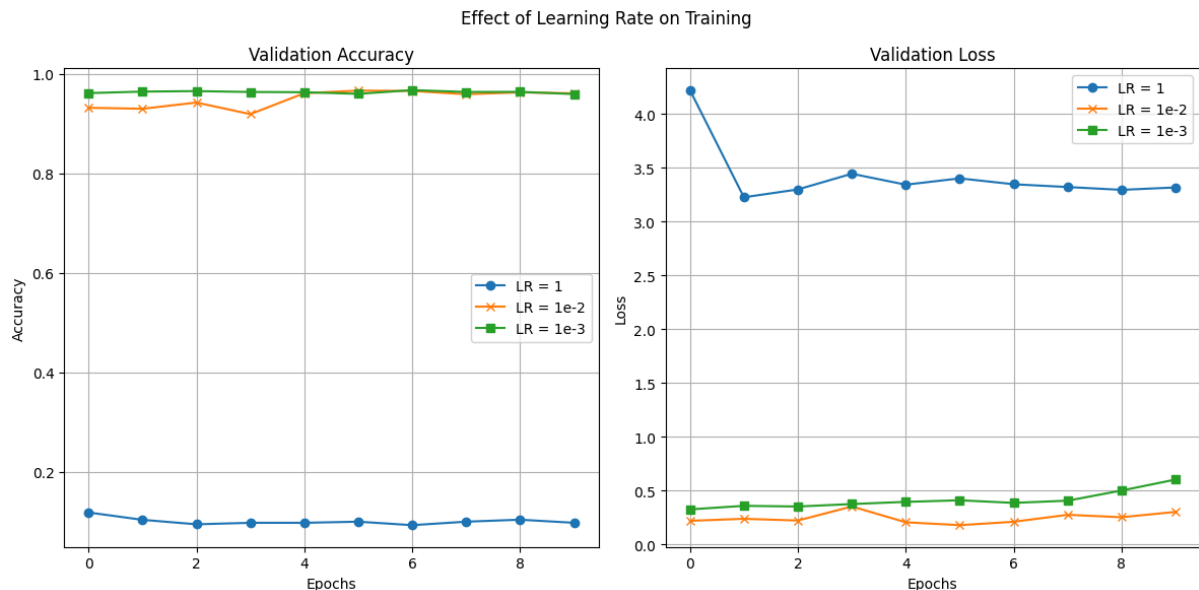
plt.legend()
plt.grid(True)

plt.suptitle("Effect of Learning Rate on Training")

```



```
plt.tight_layout()
plt.show()
```



In this experiment, the learning rate of **1e-3** gives the best overall performance. The validation accuracy improves steadily while the validation loss decreases in a stable manner. However, this does **not** mean: "Lower learning rate is always better". This experiment simply shows that, for this particular architecture and dataset, 1e-3 happens to work well.

A learning rate that is too high can create extremely unstable updates. The optimizer jumps too aggressively through the loss landscape, often overshooting the optimal region repeatedly instead of converging smoothly.

You can visualize Gradient Descent like trying to descend a mountain:

- A huge learning rate behaves like taking massive leaps downhill
- A moderate learning rate behaves like controlled running
- A tiny learning rate behaves like cautious baby steps

When the learning rate is excessively high, training may show:

- Loss fluctuating or decreasing inconsistently
- Accuracy remaining low or unstable
- Failure to converge properly

On the other hand, when the learning rate becomes too small:

- Training progresses extremely slowly
- Accuracy improves very gradually
- The optimizer may become trapped in a local minimum

The learning rate therefore controls the *speed* and *stability* of learning simultaneously.

This is why tuning the learning rate is one of the first optimization steps practitioners perform in Deep Learning. Even with a strong architecture, a poor learning rate can completely ruin training behavior. In many ways, the learning rate determines whether the Neural Network learns smoothly, or simply stumbles through the optimization process blindly.

L2 Regularization

One of the biggest problems in Deep Learning is that Neural Networks can become *too confident* in the patterns they learn. When this happens, the weights inside the network may grow excessively large as the model aggressively attempts to minimize training loss. This often leads to overfitting.

L2 Regularization belongs to a family of techniques called **Regularization Methods**, whose goal is to control model complexity and improve generalization. The core idea behind L2 is surprisingly simple: Penalize large weight values.

Instead of allowing the model to freely increase weights without consequence, L2 adds an additional penalty term to the loss function. During optimization, the Neural Network is encouraged to keep weights smaller and more controlled.

The result is usually a smoother and more stable learning process.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras import regularizers

# Define the mnist model with L2 regularization
def train_mnist_model_regularization(input_images, l2_strength):
    input_layer = layers.Input(shape=(input_images.shape
```

```
[1],))
    hidden_layer1 = layers.Dense(512, activation="relu",
                                   kernel_regularizer=regulariz
ers.l2(l2_strength))
    hidden_layer2 = layers.Dense(256, activation="relu",
                                   kernel_regularizer=regulariz
ers.l2(l2_strength))
    output_layer = layers.Dense(10, activation="softmax")

    model = keras.Sequential([input_layer, hidden_layer1,
                               hidden_layer2, output_layer])

    model.compile(optimizer="rmsprop", loss="sparse_categorical_crossentropy",
                  metrics=["accuracy"])

    return model
```

```
# Train the model on the data with l2 regularization
model_with_l2 = train_mnist_model_regularization(train_images_demo, 0.002)

history_l2 = model_with_l2.fit(train_images_demo, train_labels_demo,
                                epochs=10, batch_size=128, validation_split=0.2, verbose=1)
```

```
# Plot
plt.figure(figsize=(12, 6))

# Plot training and validation accuracy
plt.subplot(1, 2, 1)
plt.plot(history_l2.history["accuracy"], label="Training Accuracy", marker='o')
plt.plot(history_l2.history["val_accuracy"], label="Validation Accuracy", marker='x')
plt.title("Training and Validation Accuracy (L2 Regularizat
```

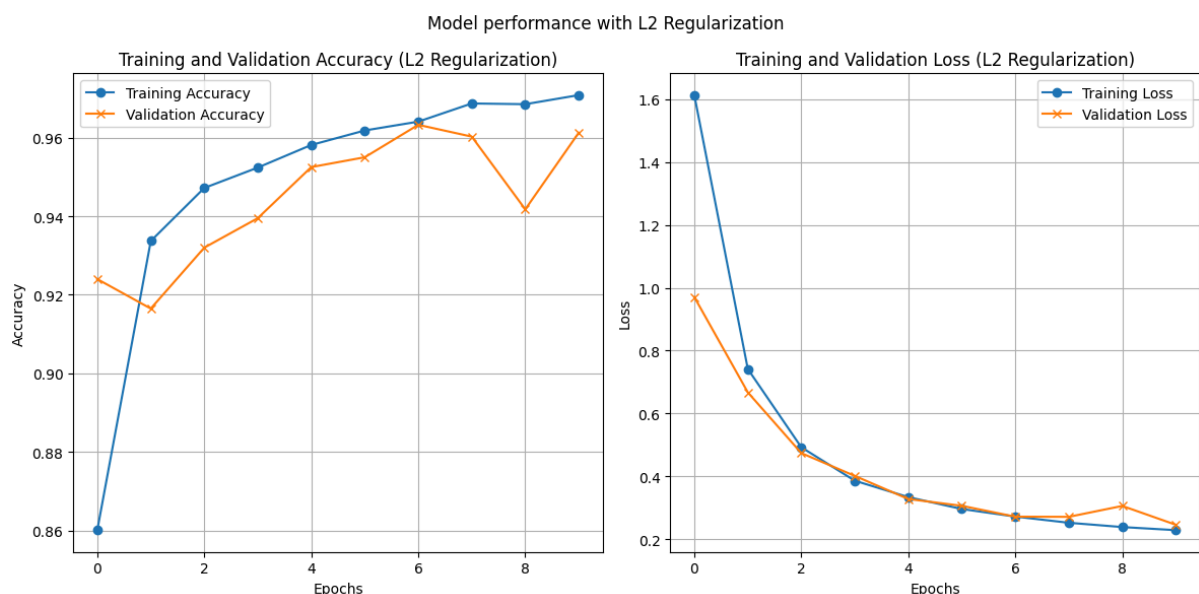
```

ion)")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

# Plot training and validation loss
plt.subplot(1, 2, 2)
plt.plot(history_l2.history["loss"], label="Training Loss",
marker='o')
plt.plot(history_l2.history["val_loss"], label="Validation
Loss", marker='x')
plt.title("Training and Validation Loss (L2 Regularizatio
n)")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.suptitle('Model performance with L2 Regularization')
plt.tight_layout()
plt.show()

```



When training with L2 Regularization, you will often notice:

- Validation loss becomes more stable
- Overfitting occurs more slowly
- The model generalizes better to unseen data

This happens because the network is prevented from becoming excessively dependent on a few extremely large weights.

You can think of L2 as a form of restraint. The model is still allowed to learn important patterns, but it is discouraged from making overly aggressive adjustments.

However, too much regularization creates another problem. If the L2 penalty becomes excessively large, the model may become *too restricted*. The weights are pushed so close to zero that the Neural Network loses learning capacity entirely. This leads to **underfitting**. The model becomes overly simple and fails to capture important relationships inside the dataset.

So once again, Deep Learning becomes a balancing act:

- Too little regularization → Overfitting
- Too much regularization → Underfitting
- Somewhere in the middle → Better generalization

Dropout

Another extremely popular regularization technique is **Dropout**. The idea behind Dropout sounds almost absurd at first: Randomly deactivate neurons during training. During each training step, certain neurons are temporarily turned off. Their outputs are ignored entirely for that iteration.

But why would we intentionally disable parts of the Neural Network? Because Neural Networks have a dangerous habit: They become overly dependent on specific neurons. Some neurons may dominate learning so strongly that the rest of the network contributes very little. This creates fragile learning behavior where the model relies too heavily on certain pathways.

Dropout prevents this dependency. By randomly removing neurons during training, the model is forced to distribute learning across many different neurons rather than depending on only a few.

```

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# Define the mnist model with Dropout
def train_mnist_model_with_dropout(input_images, dropout_rate):
    input_layer = layers.Input(shape=(input_images.shape
[1],))
    hidden_layer1 = layers.Dense(512, activation="relu")
    dropout_layer1 = layers.Dropout(dropout_rate)
    hidden_layer2 = layers.Dense(256, activation="relu")
    dropout_layer2 = layers.Dropout(dropout_rate)
    output_layer = layers.Dense(10, activation="softmax")

    model = keras.Sequential([input_layer, hidden_layer1, dropout_layer1,
                                hidden_layer2, dropout_layer2,
                                output_layer])

    model.compile(optimizer="rmsprop", loss="sparse_categorical_crossentropy",
                  metrics=["accuracy"])

    return model

```

```

# Train the model with Dropout
dropout_rate = 0.5
model_with_dropout = train_mnist_model_with_dropout(train_images_demo, dropout_rate)

history_dropout = model_with_dropout.fit(train_images_demo,
train_labels_demo,
                                         epochs=20, batch_size=128,

```

```
validation_split=0.2, verbose=1)
```

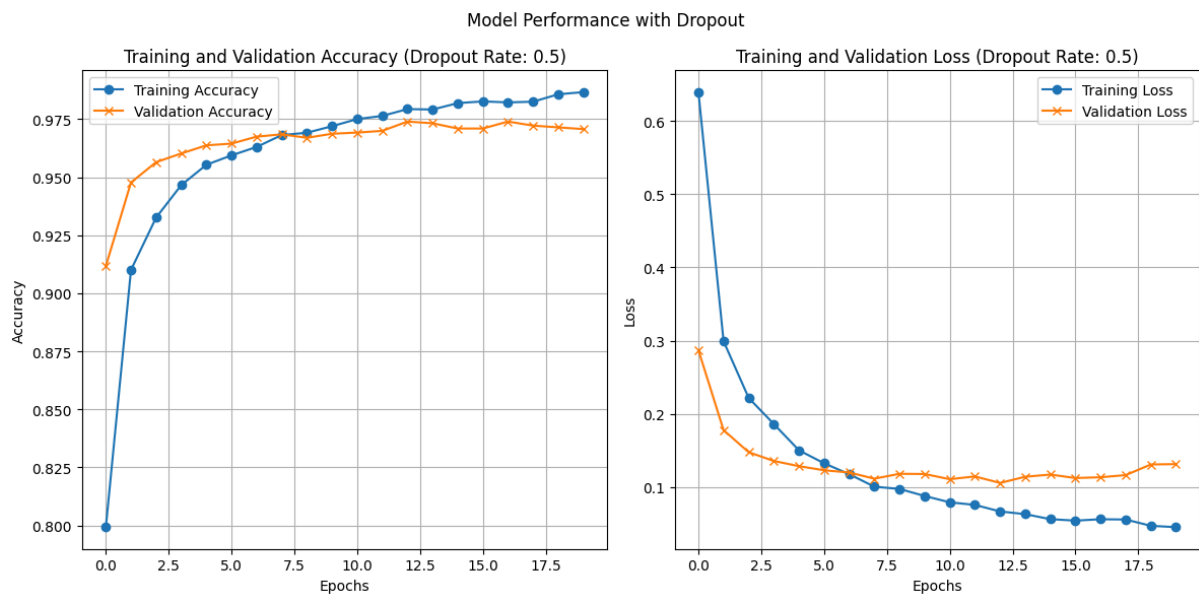
```
import matplotlib.pyplot as plt

plt.figure(figsize=(12, 6))

# Plot training and validation accuracy with Dropout
plt.subplot(1, 2, 1)
plt.plot(history_dropout.history["accuracy"], label="Training Accuracy", marker='o')
plt.plot(history_dropout.history["val_accuracy"], label="Validation Accuracy", marker='x')
plt.title(f"Training and Validation Accuracy (Dropout Rate: {dropout_rate})")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

# Plot training and validation loss with Dropout
plt.subplot(1, 2, 2)
plt.plot(history_dropout.history["loss"], label="Training Loss", marker='o')
plt.plot(history_dropout.history["val_loss"], label="Validation Loss", marker='x')
plt.title(f"Training and Validation Loss (Dropout Rate: {dropout_rate})")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.suptitle('Model Performance with Dropout')
plt.tight_layout()
plt.show()
```



As a result:

- Validation loss often becomes more stable
- Generalization improves
- Overfitting becomes less severe

You can think of Dropout like training a sports team under chaotic conditions.

If one player suddenly disappears during practice, the rest of the team must adapt and compensate. Over time, the entire team becomes more flexible and resilient.

Neural Networks behave similarly. Since neurons cannot rely on the same neighbors every iteration, they learn more robust representations.

However, there is one very important detail: Dropout is only used during training. During testing and inference, all neurons become active again. This is because randomness during prediction would make outputs unstable and inconsistent. The purpose of Dropout is to improve learning robustness during training — not to create randomness during deployment.

Early Stopping

One of the most practical techniques in Deep Learning is **Early Stopping**.

Normally, when training a Neural Network, we specify a fixed number of epochs: 10 epochs, 20 epochs, or 50 epochs. But this creates a major problem:

How do we know the model actually needs all those epochs?

In many situations, the Neural Network already reaches its best validation performance long before training finishes. After that point, the model begins memorizing the training data instead of improving generalization.

This is where Early Stopping becomes extremely useful. The idea is simple: Stop training automatically once validation performance stops improving.

Instead of blindly training until the final epoch, the model continuously monitors metrics such as validation loss. Once the validation performance becomes stable — or begins worsening — training stops automatically.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.callbacks import EarlyStopping

# Define the mnist model with Early Stopping
def train_mnist_model_with_early_stopping(input_images, patience):
    input_layer = layers.Input(shape=(input_images.shape
[1],))
    hidden_layer1 = layers.Dense(512, activation="relu")
    hidden_layer2 = layers.Dense(256, activation="relu")
    output_layer = layers.Dense(10, activation="softmax")

    model = keras.Sequential([input_layer, hidden_layer1,
                                hidden_layer2, output_layer])

    model.compile(optimizer="rmsprop", loss="sparse_categorical_crossentropy",
                  metrics=["accuracy"])

    # Define Early Stopping callback
    early_stopping_callback = EarlyStopping(monitor='val_loss',
patience=patience,
restore_best_weights=True, verbose=1)
```

```

history = model.fit(train_images_demo, train_labels_demo,
                    epochs=50, batch_size=128, validation
_split=0.2,
                    callbacks=[early_stopping_callback],
verbose=1)

# Get the epoch at which training stopped
if early_stopping_callback.stopped_epoch > 0:
    stopped_epoch = early_stopping_callback.stopped_epoch
else:
    stopped_epoch = history.epoch[-1] + 1

return model, stopped_epoch, history

```

```

# Train the model with Early Stopping
patience_value = 5
model_with_early_stopping, stopped_epoch, history_early_stopping = train_mnist_model_with_early_stopping(train_images_demo, patience_value)

print(f"Training stopped at epoch: {stopped_epoch}")

```

```

import matplotlib.pyplot as plt

plt.figure(figsize=(12, 6))

# Plot training and validation accuracy with Early Stopping
plt.subplot(1, 2, 1)
plt.plot(history_early_stopping.history["accuracy"], label="Training Accuracy", marker='o')
plt.plot(history_early_stopping.history["val_accuracy"], label="Validation Accuracy", marker='x')
plt.axvline(x=stopped_epoch - 1, color='r', linestyle='--', label=f'Stopped at Epoch {stopped_epoch}')
plt.title(f"Training and Validation Accuracy (Early Stopping, Patience: {patience_value})")
plt.xlabel("Epochs")

```

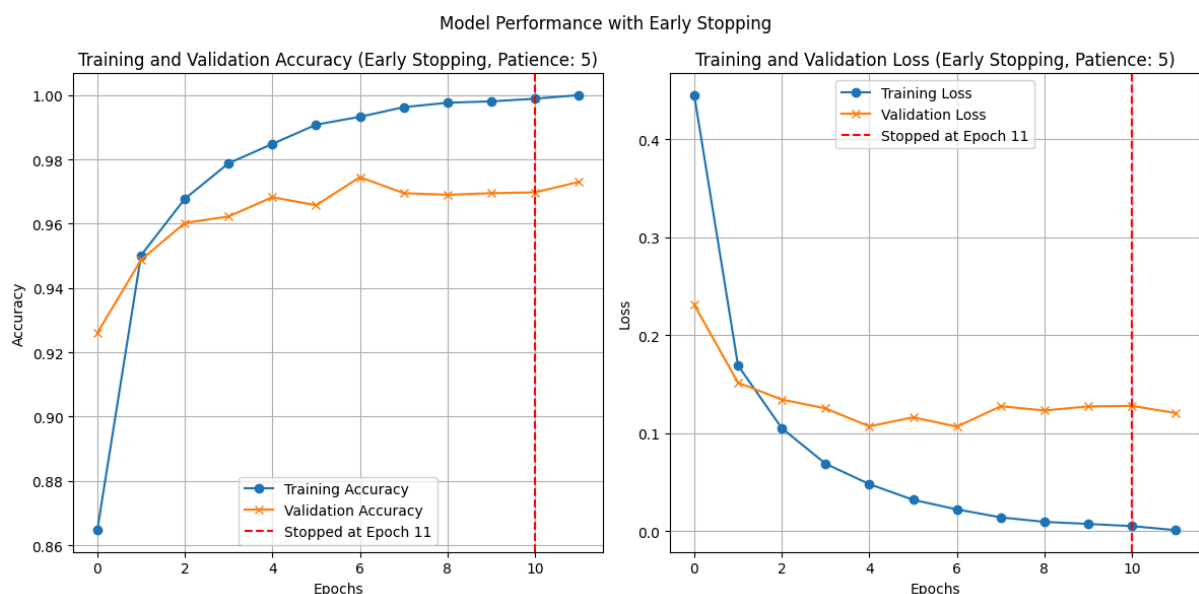
```

plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

# Plot training and validation loss with Early Stopping
plt.subplot(1, 2, 2)
plt.plot(history_early_stopping.history["loss"], label="Training Loss", marker='o')
plt.plot(history_early_stopping.history["val_loss"], label="Validation Loss", marker='x')
plt.axvline(x=stopped_epoch - 1, color='r', linestyle='--',
            label=f'Stopped at Epoch {stopped_epoch}')
plt.title(f"Training and Validation Loss (Early Stopping, Patience: {patience_value})")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

plt.suptitle('Model Performance with Early Stopping')
plt.tight_layout()
plt.show()

```



In practice, we intentionally set a very large maximum epoch value:

- 50 epochs
- 80 epochs
- Sometimes even more

But the model rarely reaches the final epoch count. Instead, training terminates exactly when validation performance suggests that learning is no longer improving meaningfully.

This makes Early Stopping significantly better than manually choosing epochs. Why? Because manual epoch selection is mostly guesswork. Early Stopping allows the model itself to determine when it has learned enough.

One of the clearest signs that Early Stopping may soon activate is this pattern:

- Training loss continues decreasing
- Validation loss stops improving or begins increasing

This usually indicates the beginning of overfitting. The model is becoming better at memorizing the training set while simultaneously becoming worse at generalizing to unseen data. Early Stopping intervenes at that moment to preserve the best version of the model.

Summary

At this point, we begin to see an important truth about Deep Learning: Building a Neural Network is easy. Training it properly is the difficult part.

A Neural Network can easily memorize noise, random labels, or meaningless relationships if we allow it to. Because of this, much of Deep Learning revolves around controlling *how* the model learns rather than simply increasing model size.

In this section, we explored several important optimization and regularization techniques:

- **Noise Features** showed how meaningless inputs damage generalization and encourage spurious correlations.
- **Random Labels** demonstrated that Neural Networks can memorize almost anything, even when the data itself makes no sense.
- **Learning Rate Tuning** revealed how optimization stability depends heavily on update step size.

- **L2 Regularization** penalized excessively large weights to reduce overfitting.
- **Dropout** prevented the network from depending too heavily on specific neurons.
- **Early Stopping** automatically halted training once validation performance stopped improving.

All these techniques share one common purpose: Improve generalization.

Because ultimately, the goal of Deep Learning is not to memorize the training set perfectly. The goal is to learn patterns that remain useful when the model encounters completely new data. That is the difference between a Neural Network that merely remembers, and a Neural Network that truly learns.

References

<https://www.geeksforgeeks.org/machine-learning/training-vs-testing-vs-validation-sets/>

<https://www.geeksforgeeks.org/deep-learning/training-and-validation-loss-in-deep-learning/>

<https://www.kaggle.com/code/sid321axn/regularization-techniques-in-deep-learning>

<https://www.geeksforgeeks.org/deep-learning/dropout-regularization-in-deep-learning/>

https://d2l.ai/chapter_multilayer-perceptrons/dropout.html

<https://www.geeksforgeeks.org/machine-learning/regularization-by-early-stopping/>

<https://machinelearningmastery.com/early-stopping-to-avoid-overtraining-neural-network-models/>

Next Section:

10. Deploying Neural Network Models